Interactive 8-Puzzle Problem Solver

Uniform Cost Search - Algorithm 5

Student: Arush Barure
Registration number: 24BDS0331
Technology: HTML5, CSS3, JavaScript ES6 and Bootstrap 5.3.3

Submission links

Repository: https://github.com/arush-Barure/AI-8Puzzle-Visualizer

GitHub Pages: https://arush-barure.github.io/AI-8Puzzle-Visualizer/

README: Read project documentation

Demonstration video: Watch/download the narrated demonstration

Scope and interpretation

The allocation table assigns registration 24BDS0331 to algorithm 5: Uniform Cost Search. The brief explicitly instructs each student to implement only the allocated algorithm. Therefore, this project implements UCS only. Its comparison section explains BFS, DFS, DLS, IDDLS, GBFS and A* theoretically; it does not claim fabricated runtime measurements for those algorithms.

Problem model

A state is an ordered array of nine values: the eight tiles and 0 for the blank. Each action swaps the blank with an adjacent tile. There are at most four successors, and each move costs 1. The objective is to reach the user-defined goal with minimum total cost. For this unit-cost problem, minimum cost is the minimum number of moves.

Algorithm design

Working principle

UCS repeatedly removes the least-cost node from a binary minimum heap. Its evaluation function is f(n) = g(n), with no heuristic. Equal-cost nodes use insertion order for deterministic tie-breaking. A goal is accepted only when removed from the frontier, preserving optimality. Best-known-cost and closed-state maps avoid redundant expansion. Parent pointers reconstruct the solution path.

Step	Operation
1	Validate both permutations and compare inversion parity.
2	Push the initial state at g = 0.
3	Pop the least-cost unsettled state; mark it explored.
4	If it is the goal, reconstruct the path and stop.
5	Generate legal blank moves at g + 1; queue improved states.
6	Repeat until solved or the frontier is empty.

Completeness, optimality and complexity

UCS is complete for finite branching with step costs bounded below by a positive value, and is optimal for nonnegative costs. This puzzle has finite state space and unit positive costs. A general tree-search bound is O(b^(1 + floor(C*/epsilon))) for time and space; b is branching, C* is optimal path cost, and epsilon is the smallest step cost. A heap-based finite graph implementation has O((V + E) log V) search time and O(V + E) storage for the general lazy-entry queue. Only 181,440 permutations are reachable from a fixed 3 x 3 goal.

Advantages and limitations

Advantages: guarantees the least-cost path; needs no heuristic; handles arbitrary valid goal states; exposes a clear cost-based expansion order. Limitations: explores many states without goal guidance; frontier and recorded playback may use substantial memory. High playback speed batches expansions to keep interactive controls usable.

Heuristic context

Misplaced tiles counts nonblank tiles outside their goal positions. Manhattan distance sums row and column displacement for nonblank tiles. These heuristics belong to informed search methods such as GBFS and A*. Neither is used by UCS. With equal edge costs, UCS visits the same depth layers as BFS.

Validation & demonstration

Verified test cases

Case	Expected result
Already at goal	0 moves; root recognized on removal.
One legal move from goal	Optimal cost 1.
Default input: 1 2 3 / 4 0 6 / 7 5 8	Optimal cost 2; every path transition is legal.
PDF example: 2 8 3 / 1 6 4 / 7 0 5; goal: 1 2 3 / 8 0 4 / 7 6 5	Optimal cost 5 with a nonstandard goal.
Swap tiles 1 and 2 in the standard goal	Unsolvable, rejected before search.
All 90 states within six moves of standard goal	UCS matches independently enumerated shortest distances.
100 random walks from an arbitrary goal	Every generated state preserves goal parity.

Statistics definitions

Generated nodes counts the initial node plus every legal successor, including duplicates. Expanded nodes counts valid frontier removals, including the goal. Maximum depth includes discovered nodes. Search CPU time accumulates search steps and excludes animation delays and rendering. Memory is an explicitly approximate solver-storage model, 160 bytes per peak stored record/reference unit; browser, DOM and playback-history memory are excluded.

Suggested demonstration sequence

Introduce the assignment and UCS. Run the easy example and inspect open/closed lists and successors. Pause, resume, step forward and backward, then animate the final path. Load the PDF example and solve it. Explain g(n), unit costs and the measured summary. Generate a solvable puzzle. Enter an invalid permutation and an unsolvable pair. Finish with theoretical algorithm comparison, heuristic context and project structure.

References and resource acknowledgment

UC Berkeley CS188: Uninformed Search and Informed Search. Bootstrap 5.3.3, MIT license, locally vendored stylesheet. GitHub Docs: Configuring a publishing source for GitHub Pages. Source assignment: Artificial Intelligence QP 1.pdf. Implementation and documentation prepared with OpenAI Codex assistance; search logic is written from scratch without AI/search libraries. The student should review the code and demonstrate their understanding.